GV JAVA CLI PROJECT SERIES

PROJECT REQUIREMENTS — V2.0

GearVault Equipment Rental

A menu-driven CLI application for managing an equipment and tool rental business — cataloging inventory by category, tracking active rental agreements and returns, logging maintenance, and generating utilization and revenue reports. Built with Java OOP, Spring Boot, and MongoDB.

JAVA OOP | SPRING BOOT | MONGODB | MENU-DRIVEN | NO WEB LAYER

4 COLLECTIONS

3 EQUIPMENT SUBTYPES

8 SECTIONS

3 RENTAL RATE STRATEGIES

01 / OVERVIEW

Project Overview

Application Summary

GearVault is a menu-driven command-line application built with Java, Spring Boot, and MongoDB. It manages an equipment and tool rental business — cataloging every item in inventory, tracking who has it out and when it's due back, logging maintenance work, and computing utilization and revenue on demand. The rental operator checks equipment out to a customer, records the return, and the system flags overdue rentals, equipment due for service, and warranties about to lapse.

The architecture follows the same four-collection pattern established across the CLI series: users, equipment, rental agreements, and maintenance logs are all separate top-level documents. Rental agreements reference their equipment by `equipmentId`. Maintenance logs reference their equipment the same way, enabling cross-equipment aggregation without loading equipment documents. Rental rate strategies provide the core OOP design surface, with each equipment subtype pricing a rental differently based on how that category is actually rented in the field.

Technology Stack

Spring Boot	Entry point via `CommandLineRunner`. No web layer. Spring DI wires all services. `SessionContext` holds the current logged-in user.
Spring Data MongoDB	Four repositories: `UserRepository`, `EquipmentRepository`, `RentalAgreementRepository`, and `MaintenanceLogRepository`. Rental agreements reference equipment by `equipmentId`. Polymorphic `_class` discriminator for equipment subtypes in the equipment collection.
Java OOP	Abstract `Equipment` with three concrete subtypes each implementing `computeRentalCost()` differently. Strategy pattern for rental pricing — each subtype instantiates its own `RentalRateStrategy` in the no-arg constructor; field is `@Transient`. Factory pattern for equipment creation. Chain of responsibility for alerts.
Menu System	Two-level numbered menus. `IMenu` interface with `show()` and `printOptions()` — all menu classes implement it directly. `Scanner`-based `InputHandler` with debug mode reading from `test_input.txt`. `do-while` loop navigation. `0` always means Back or Exit.
BCrypt	Passwords hashed via Spring Security Crypto.

02 / DOMAIN

Domain Model

Equipment Hierarchy

All equipment types extend the abstract `Equipment` class. The key design surface is `computeRentalCost(int days)` — each subtype implements a different pricing model reflecting how that category of equipment is actually rented.

Equipment ABSTRACT — computeRentalCost(int) · getEquipmentType() · getSummary()

PowerTool	HeavyMachinery	EventEquipment
batteryPowered (boolean)	engineHours	eventCategory (enum: TENT, TABLE_CHAIR, LIGHTING, AUDIO, OTHER)
voltage	requiresOperatorCert (boolean)	quantityAvailable
toolClass (enum)	fuelType (enum)	setupRequired
hourlyRate (double)	dailyRate (double)	flatEventRate (double)
dailyRate (double) — daily cap	depositAmount (double)	List<BookingNote> setupLogs
requiresChargeCheck()	getCertWarning() — formatted string	Pricing: flat per-event rate scaled by quantity
Pricing: hourly-with-daily-cap — low friction, short-term rentals	Pricing: tiered daily rate + deposit — declines per extra day	e.g. tents, tables/chairs, PA systems, stage lighting
e.g. drill, circular saw, sander, nail gun	e.g. excavator, generator, skid steer, compressor	

Core Classes

`User`	id, name, username, hashedPassword, isAdmin (boolean, default false), overdueGraceDays (int, default 1). Top-level MongoDB document.
`Equipment (abstract)`	id, userId, name, brand, model, serialNumber, purchasePrice, purchaseDate, warrantyExpiry (LocalDate, optional), condition (enum), status (enum: AVAILABLE, RENTED, IN_MAINTENANCE, RETIRED), notes. Abstract: `computeRentalCost(int days)`, `getEquipmentType()`. Own MongoDB collection stored polymorphically via `_class` discriminator.
`RentalAgreement`	Separate top-level MongoDB document. equipmentId → equipment (kept for lookups against live equipment state — status, condition, alerts), userId → users (denormalized copy of the equipment owner's userId, set at creation so agreements can be queried directly by user without loading equipment). Point-in-time snapshot of the equipment at checkout — equipmentName, equipmentSerialNumber, equipmentBrand, equipmentModel — copied once at creation so the agreement reads correctly even if the equipment is later renamed, retired, or deleted; these are not re-synced if the source equipment changes, the same way a receipt doesn't update if the store re-labels the product. customerName, customerPhone, checkoutDate, dueDate, returnDate (nullable), depositAmount, totalCost (computed at checkout via the equipment's rate strategy), status (enum: ACTIVE, RETURNED — "overdue" is a computed comparison against `dueDate`, not a persisted state; see RentalStatus enum note).
`BookingNote`	Embedded in `EventEquipment`. eventDate, setupDetails, headcount, notes. Selected by ArrayList index when needed. Never a top-level document.
`MaintenanceLog`	Separate top-level MongoDB document. equipmentId → equipment (kept for the same reason as `RentalAgreement`), userId → users (same denormalization rationale as `RentalAgreement`). Point-in-time snapshot of the equipment at service time — equipmentName, equipmentSerialNumber, equipmentBrand, equipmentModel — copied once at creation, same rationale as `RentalAgreement`. serviceDate, serviceType (enum), description, cost, hoursAtService, technicianName. Supports full CRUD from the Maintenance menu. Multiple records per equipment item.

`RevenueReport`	Transient. Generated on demand — never persisted. Fields: `totalRevenue` (double, scalar), `revenueByType` (`Map<EquipmentType, Double>`, one entry per equipment type), `depositOutstanding` (double, scalar — sum of `depositAmount` across active agreements), `topEarningItems` (`List<RevenueLineItem>`, ranked descending, capped at 5). Formatted for display by `ReportMenu`.
`RevenueLineItem`	Transient. Small value object used only inside `topEarningItems`: `equipmentName` (String), `equipmentType` (enum), `revenue` (double). Never a top-level document.
`UtilizationReport`	Transient. Fields: `totalEquipment`, `totalAvailable`, `totalRented` (all int, scalar), `utilizationRate` (double, scalar — fleet-wide), `utilizationByType` (`Map<EquipmentType, Double>`, one rate per equipment type). Generated by `ReportService`.
`AlertResult`	Transient value object returned by each alert strategy. Fields: `Equipment equipment`, `AlertType alertType`, `String message`. Carries the triggering item and structured type alongside the human-readable message so `AlertMenu` can display rich, contextual information.
`AlertManager`	Spring `@Component`. Auto-populated `List<AlertStrategy>`. Single `evaluate(List<Equipment>)` traverses all equipment and strategies, returning `List<AlertResult>`. Called exclusively from `AlertMenu` — never from service classes.
`SessionContext`	Spring `@Component` singleton. Holds the currently logged-in `User` via a static field, set at login and cleared at logout. Exposes `SessionContext.getUser()` as a static method so plain objects (strategies, factories) can access the current user without injection. `OverdueReturnStrategy` calls `SessionContext.getUser().getOverdueGraceDays()`.

Enumerations

EquipmentType

`POWER_TOOL` `HEAVY_MACHINERY` `EVENT_EQUIPMENT`

EquipmentStatus

`AVAILABLE` `RENTED` `IN_MAINTENANCE` `RETIRED`

ToolClass

`DRILL` `SAW` `SANDER` `NAIL_GUN` `OTHER`

FuelType

`DIESEL` `GASOLINE` `ELECTRIC` `PROPANE`

EventCategory

`TENT` `TABLE_CHAIR` `LIGHTING` `AUDIO` `OTHER`

EquipmentCondition

`EXCELLENT` `GOOD` `FAIR` `POOR` `NEEDS_REPAIR`

RentalStatus

`ACTIVE` `RETURNED`

No `OVERDUE` value — overdueness is a computed comparison (`dueDate` + `overdueGraceDays` vs. today) evaluated at read time by `OverdueReturnStrategy` and "View overdue rentals," not a status that gets persisted. There's no scheduler in this app to transition it, so a stored value would either need a lazy write-on-read (a side effect hiding in a getter) or would silently go stale.

ServiceType

`CLEANING` `BLADE_BIT_REPLACEMENT` `ENGINE_SERVICE` `CALIBRATION` `SAFETY_INSPECTION` `REPAIR` `OTHER`

AlertType

`OVERDUE_RETURN` `MAINTENANCE_DUE` `WARRANTY_EXPIRING`

03 / OOP

Object-Oriented Design

Design Requirements

INHERITANCE
Abstract Equipment — 3 concrete subtypes
Each overrides computeRentalCost(int days)
Each overrides getEquipmentType()
Constructor chaining via super()

INTERFACES
AlertStrategy — supports(), evaluate() → AlertResult
RentalRateStrategy<T> — computeCost(T equipment, int days)
Summarizable — getSummary()

ENCAPSULATION
No direct field access
Rental cost calculated, not stored until checkout
Booking notes via addBookingNote() / editBookingNote(int index)
Password BCrypt hashed only

PATTERNS
Strategy — rental pricing per equipment subtype
Chain of Responsibility — AlertManager
Factory — EquipmentFactory
IMenu interface — show(), printOptions()

Rental Rate Strategies

Rate fields live on the subtype that actually needs them — `hourlyRate` / `dailyRate` on `PowerTool`, `dailyRate` / `depositAmount` on `HeavyMachinery`, `flatEventRate` on `EventEquipment` — since they aren't shared across categories the way `purchasePrice` is. Because of that, `RentalRateStrategy<T extends Equipment>` is generic and each strategy is handed the whole subtype-specific object rather than a single generic rate: `double computeCost(T equipment, int days)`. Each equipment subtype initializes its own strategy in the no-arg constructor (e.g. `this.rateStrategy = new HourlyCappedRateStrategy()`) and declares the field typed to itself, e.g. `RentalRateStrategy<PowerTool> rateStrategy` on `PowerTool`. Since MongoDB deserializes via the no-arg constructor, the strategy is always wired whether the object is freshly created by `EquipmentFactory` or loaded from the database — no post-load wiring needed. The field is annotated `@Transient` so it is never persisted. `computeRentalCost(int days)` on each subtype delegates to `rateStrategy.computeCost(this, days)`, passing itself so the strategy can read its own rate fields directly.

Subtype	Strategy	Formula	Example
`PowerTool`	`HourlyCappedRateStrategy`	min(equipment.hourlyRate × 8, equipment.dailyRate) × days, with a 1-day minimum	$8/hr drill, $35/day cap, rented 3 days = min(64, 35) × 3 = $105
`HeavyMachinery`	`TieredDailyRateStrategy`	equipment.dailyRate (day 1) + (days – 1) × 0.80 × equipment.dailyRate + equipment.depositAmount	$220/day excavator, 4 days = $220 + (3 × $176) = $748 + deposit
`EventEquipment`	`FlatEventRateStrategy`	equipment.flatEventRate × equipment.quantityAvailable	$20/bundle rate × 8 bundles available = $160

Alert Strategies

`OverdueReturnStrategy`	`supports()` — true for any non-null `Equipment` currently referenced by an `ACTIVE` rental agreement. `evaluate()` — fires when today's date is past the agreement's `dueDate` plus the current user's `overdueGraceDays`; returns an `AlertResult` with `AlertType.OVERDUE_RETURN`, the equipment, and a message showing days overdue. Requires `RentalAgreementService` to look up the active agreement. Only invoked from `AlertMenu`.
`MaintenanceDueStrategy`	`supports()` — true for any non-null equipment. `evaluate()` — fires when no maintenance log exists and the equipment is more than 6 months old, or when the most recent service is more than 6 months old (or, for `HeavyMachinery`, when `engineHours` exceeds the `hoursAtService` value from that equipment's most recent `MaintenanceLog` by more than 250 hours — there is no separately stored threshold field); returns an `AlertResult` with `AlertType.MAINTENANCE_DUE` and a message indicating last service date or "never serviced." Requires `MaintenanceLogService`. Only invoked from `AlertMenu`.
`WarrantyExpiringStrategy`	`supports()` — true for any equipment with a non-null `warrantyExpiry`. `evaluate()` — fires when the warranty expiry date is within 30 days of today; returns an `AlertResult` with `AlertType.WARRANTY_EXPIRING` and a message showing the expiry date. Only invoked from `AlertMenu`.

Key Design Notes

Four Collections	Users, equipment, rental agreements, and maintenance logs are all separate top-level documents. Rental agreements and maintenance logs both reference their item by `equipmentId` rather than embedding — this keeps the equipment document small and allows either collection to be queried independently (e.g. all active rentals, or all maintenance history) without loading equipment.
Rental Rate Strategy	Rate fields (`hourlyRate`, `dailyRate`, `depositAmount`, `flatEventRate`) live directly on the subtype that charges them, not on the abstract `Equipment` class, since they aren't shared across categories. `RentalRateStrategy<T extends Equipment>` is generic so each strategy receives the whole subtype-specific object and reads its own fields directly, rather than the interface trying to guess which single "base rate" to pass. Each subtype initializes its own strategy in the no-arg constructor. MongoDB deserializes via the no-arg constructor, so the strategy is always set on load without any post-load hook. The field is `@Transient` and never stored. `computeRentalCost(int days)` on each subtype delegates to `rateStrategy.computeCost(this, days)`.
EquipmentFactory	`EquipmentFactory.create(EquipmentType)` prompts common fields first (name, brand, model, serial number, purchase price, purchase date, condition, optional warranty expiry and notes), then type-specific fields, including the rate fields each subtype's strategy needs. Power tools prompt for voltage, battery flag, tool class, hourly rate, and daily rate cap. Heavy machinery prompts for engine hours, cert requirement, fuel type, daily rate, and deposit amount. Event equipment prompts for category, quantity available, setup-required flag, and flat event rate.
BookingNote Selection	When viewing or editing setup notes on `EventEquipment`, records are listed by numbered ArrayList index. The user selects by number — no UUID needed. The list is stable for the duration of the menu interaction.
Status & Checkout	An equipment item's `status` is set to `RENTED` when a rental agreement is created and reverts to `AVAILABLE` when the agreement's return is recorded. Attempting to check out an item that is not `AVAILABLE` is rejected with a clear message pointing to its current status.

MaintenanceLog	Full top-level documents in their own collection, same pattern as `MaintenanceRecord` in the Auto Warehouse project. This allows cross-equipment maintenance reports (total spend, cost by service type) without loading every equipment document. `MaintenanceLogRepository` exposes `findByEquipmentId()` and `findByUserIdOrderByServiceDateDesc()`.
Equipment Snapshot	`RentalAgreement` and `MaintenanceLog` both copy equipmentName, equipmentSerialNumber, equipmentBrand, and equipmentModel at creation time, straight from the `Equipment` object already in hand — no extra query. This is a point-in-time snapshot, not a cache: the point is that a rental or service record should always read as it did at that moment, the same way a receipt still shows the product name and price as they were at checkout even after the store re-labels or discontinues the item. These fields are never re-synced if the source equipment is later renamed. `equipmentId` is kept alongside the snapshot for anything that needs the equipment's *current* state — status, condition, alert evaluation — rather than its identity at that point in time.

04 / MENUS

Menu System

Main Menu

```
GearVault – Equipment Rental Manager  //  v2.0

  1  Equipment                          catalog and manage rental inventory
  2  Rentals                            check out, check in, and track agreements
  3  Maintenance                        log and review service records
  4  Reports                            revenue and utilization
  5  Alerts                             overdue, maintenance, and warranty reminders
  6  Settings                           account preferences
  0  Exit

> Select option:
```

Submenus

1. Equipment

1 List all equipment
2 List available equipment
3 Add equipment
4 View equipment detail
5 Edit equipment
6 Add booking note
7 Edit booking note
8 Retire equipment
0 Back

2. Rentals

1 Check out equipment
2 Check in / return
3 View active rentals
4 View overdue rentals
5 View rental history
0 Back

3. Maintenance

1 Log maintenance record
2 View history by equipment
3 View all recent records
4 Delete record
0 Back

4. Reports

1 Revenue report
2 Utilization report
3 Overdue report
4 Maintenance cost by type
0 Back

5. Alerts

1 View all alerts
2 Overdue return alerts
3 Maintenance due alerts
4 Warranty expiring alerts
0 Back

6. Settings

1 Change password
2 Set overdue grace days
3 Add user (admin only)
4 Delete user (admin only)
0 Back

05 / FEATURES

Feature Specifications

Equipment

List all equipment	Display all equipment: name, brand/model, type badge, condition, status (AVAILABLE / RENTED / IN_MAINTENANCE / RETIRED), and purchase price. Sorted by name ascending, filterable by type or status.
List available equipment	Filtered view showing only items with `status = AVAILABLE`. Used when checking out a new rental. Displays name, type, condition, and the subtype's rate field(s) (e.g. hourly/daily rate for power tools, daily rate for machinery, flat event rate for event equipment).
Add equipment	Select equipment type first (POWER_TOOL / HEAVY_MACHINERY / EVENT_EQUIPMENT). `EquipmentFactory` prompts common then type-specific fields. New items default to `status = AVAILABLE`.
View equipment detail	Full item fields, purchase price, current status, and type-specific detail (voltage and tool class for power tools, engine hours for machinery, category and quantity for event equipment). Most recent 3 maintenance records and rental history shown inline.
Edit equipment	Update condition, notes, warranty expiry, and type-specific editable fields — voltage, hourly rate, and daily rate for power tools; engine hours, daily rate, and deposit amount for machinery; quantity available and flat event rate for event equipment. Serial number, brand, model, and purchase date are immutable once set.
Add booking note	Event equipment only. Select item from numbered list filtered to `EventEquipment`. Prompt event date, setup details, headcount, and optional notes. Appends to the item's embedded `setupLogs` list.
Edit booking note	Event equipment only. Select item from numbered list filtered to `EventEquipment`, then select a note from its `setupLogs` list by numbered ArrayList index (see BookingNote Selection design note). Prompt updated event date, setup details, headcount, and notes; overwrites the selected entry in place.
Retire equipment	Confirm before retiring. Rejected if the item is currently `RENTED` — must be returned first. Sets `status = RETIRED` rather than hard-deleting, preserving rental and maintenance history for reporting.

Rentals

Check out equipment	Select equipment from the available list. If the selected item is `HeavyMachinery` with `requiresOperatorCert = true`, print `getCertWarning()` and require confirmation before proceeding. Prompt customer name, phone, and due date. Deposit amount pre-fills from the equipment's `depositAmount` field when present (`HeavyMachinery`) or defaults to zero otherwise, and can be overridden before saving. `totalCost` is computed immediately via the equipment's `computeRentalCost(days)` and stored on the agreement, along with a snapshot of the equipment's name, serial number, brand, and model at that moment. Sets equipment `status = RENTED`.
Check in / return	Select active rental from numbered list. Prompt actual return condition. Sets `returnDate`, updates agreement `status = RETURNED`, and reverts equipment `status = AVAILABLE`. If returned after `dueDate`, displays the number of overdue days.
View active rentals	All agreements with `status = ACTIVE`, sorted by due date ascending. Displays the stored equipment name/brand/model snapshot alongside customer, checkout date, due date, and days remaining (or overdue, in red-flag styling) — no live equipment lookup needed.
View overdue rentals	Filtered view of active agreements past their due date plus grace period. Same source data as the `OverdueReturnStrategy` alert, surfaced here as a direct list for quick follow-up calls.
View rental history	All agreements for the current user, sorted by checkout date descending, including returned ones. Displays the stored equipment name/brand/model snapshot alongside customer, dates, and total cost — reflects what was rented even if that equipment has since been renamed or retired. Useful for customer dispute resolution.

Maintenance

Log maintenance record	Select equipment from numbered list. Prompt service date, service type (from enum), description, cost, hours at service, and technician name. Persists as a new `MaintenanceLog` document referencing `equipmentId`, along with a snapshot of the equipment's name, serial number, brand, and model at that moment. Optionally sets equipment `status = IN_MAINTENANCE` for the duration.
View history by equipment	Select equipment from numbered list. Display all maintenance logs sorted by service date descending. Show service type badge, date, description, cost, and technician. Running cost total at the bottom.
View all recent records	All maintenance logs for the current user, sorted by service date descending. Displays the stored equipment name snapshot directly from each record — no batch equipment lookup needed. Useful for reviewing recent shop activity across the fleet.
Delete record	Select equipment, then record from numbered list. Confirm before deletion. No cascading effects.

Reports

Revenue report	Displays `RevenueReport`: total rental revenue from completed and active agreements, revenue by equipment type as an ASCII bar chart rendered by `BarChartUtil`, total deposit amount currently outstanding on active rentals, and the top 5 earning items by total revenue.
Utilization report	Total equipment count, count currently rented vs. available, overall utilization rate (%), and utilization rate by equipment type. Sorted by utilization rate descending — surfaces which categories are underused.
Overdue report	All currently overdue rentals with days overdue, customer contact info, and estimated late fees (if configured). Sorted by days overdue descending.
Maintenance cost by type	Aggregated maintenance spend grouped by `ServiceType` enum. Ranked by total cost descending. Counts and totals per type, with an ASCII bar chart. Covers all equipment for the logged-in user.

Settings

Change password	Prompt current password for verification, then prompt new password twice to confirm match. Hash and persist via `UserRepository`.
Set overdue grace days	Prompt a new integer grace period for the overdue return alert. Validates it is zero or positive. Persists to the current user document and updates `SessionContext` in place so subsequent alert evaluations reflect the change immediately.
Add user (admin only)	Hidden from non-admin users. Prompt name, username, password, and admin flag. Hash password before persisting. Username must be unique — reject duplicates.
Delete user (admin only)	Hidden from non-admin users. Select user from numbered list (excludes current admin). Confirm before deletion. Hard deletes in order: all maintenance logs → all rental agreements → all equipment → user document.

06 / ARCHITECTURE

Architecture & Implementation

Package Structure

```
io.gearvault
├── GearVaultApplication.java          // Spring Boot entry point
├── menu/
│   ├── MainMenu.java
│   ├── EquipmentMenu.java
│   ├── RentalMenu.java
│   ├── MaintenanceMenu.java
│   ├── ReportMenu.java
│   ├── AlertMenu.java
│   └── SettingsMenu.java
├── model/
│   ├── User.java
│   ├── Equipment.java                 // abstract
│   ├── PowerTool.java
│   ├── HeavyMachinery.java
│   ├── EventEquipment.java
│   ├── BookingNote.java               // embedded in EventEquipment
│   ├── RentalAgreement.java           // top-level document
│   ├── MaintenanceLog.java            // top-level document
│   ├── RevenueReport.java             // transient
│   ├── RevenueLineItem.java           // transient — used in RevenueReport.topEarningItems
│   └── UtilizationReport.java         // transient
├── enums/
│   ├── EquipmentType.java
│   ├── EquipmentStatus.java
│   ├── ToolClass.java
│   ├── FuelType.java
│   ├── EventCategory.java
│   ├── EquipmentCondition.java
│   ├── RentalStatus.java
│   ├── ServiceType.java
│   └── AlertType.java
```

```
├── interfaces/
│   ├── IMenu.java                    // show(), printOptions()
│   ├── AlertStrategy.java
│   ├── RentalRateStrategy.java       // generic<T extends Equipment> — computeCost(T equip
│   └── Summarizable.java
├── pricing/
│   ├── HourlyCappedRateStrategy.java      // implements RentalRateStrategy<PowerTool>
│   ├── TieredDailyRateStrategy.java       // implements RentalRateStrategy<HeavyMachinery>
│   └── FlatEventRateStrategy.java         // implements RentalRateStrategy<EventEquipment>
├── alert/
│   ├── AlertManager.java
│   ├── AlertResult.java              // transient — Equipment, AlertType, message
│   ├── OverdueReturnStrategy.java
│   ├── MaintenanceDueStrategy.java
│   └── WarrantyExpiringStrategy.java
├── factory/
│   └── EquipmentFactory.java
├── repository/
│   ├── UserRepository.java
│   ├── EquipmentRepository.java      // findByUserId(), findByUserIdAndStatus()
│   ├── RentalAgreementRepository.java // findByEquipmentId(), findByUserIdAndStatus()
│   └── MaintenanceLogRepository.java  // findByEquipmentId(), findByUserIdOrderByServiceDat
├── service/
│   ├── UserService.java
│   ├── EquipmentService.java
│   ├── RentalAgreementService.java
│   ├── MaintenanceLogService.java
│   └── ReportService.java
├── authentication/
│   ├── AuthenticationService.java
│   └── SessionContext.java           // Spring @Component, static getUser() — holds logged
└── util/
    ├── InputHandler.java             // static methods — debug mode reads from test_input.
    └── BarChartUtil.java             // ASCII bar chart for revenue and utilization report
```

MongoDB Document Mapping

Collection	Aggregate Root	Embedded	Notes
`users`	User	—	Single user type. Fields include `isAdmin` (boolean, default false) and `overdueGraceDays` (int, default 1) for alert configuration.
`equipment`	Equipment (+ subtypes)	List<BookingNote> (EventEquipment only)	Polymorphic via `_class`. userId → users. BookingNotes embedded in EventEquipment — selected by ArrayList index, no UUID needed.
`rental_agreements`	RentalAgreement	—	equipmentId → equipment · userId → users. Separate collection so active/overdue rentals can be queried without loading equipment documents.
`maintenance_logs`	MaintenanceLog	—	equipmentId → equipment · userId → users. Separate collection enables cross-equipment aggregation (total fleet maintenance spend, cost by service type) without loading equipment documents.

07 / ALERTS

Alert System — Chain of Responsibility

All alert strategies are registered in a central `AlertManager` as a Spring-managed `List<AlertStrategy>`. `AlertMenu` calls `alertManager.evaluate(equipment)` passing a `List<Equipment>` — the manager iterates every item against every strategy, each deciding via `supports()` whether it applies. Each strategy returns an `AlertResult` carrying the triggering `Equipment`, an `AlertType` enum value (with a `displayName` field), and a human-readable message. Adding a new alert type requires zero changes to any service class — implement `AlertStrategy`, annotate with `@Component`, and Spring auto-registers it.

`AlertStrategy.java` · `AlertResult.java` · `AlertType.java` · `AlertManager.java`

Alert System — Chain of Responsibility — JAVA

```java
// AlertStrategy.java — single interface for all alert strategies
public interface AlertStrategy {
    boolean supports(Equipment equipment);
    AlertResult evaluate(Equipment equipment);
}

// AlertType.java — enum with displayName for rendering in AlertMenu
public enum AlertType {
    OVERDUE_RETURN("Overdue Return"),
    MAINTENANCE_DUE("Maintenance Due"),
    WARRANTY_EXPIRING("Warranty Expiring");

    private final String displayName;
    AlertType(String displayName) { this.displayName = displayName; }
    public String getDisplayName() { return displayName; }
}

// AlertResult.java — structured result carrying equipment, type, and message
public class AlertResult {
    private final Equipment equipment;
    private final AlertType   alertType;
    private final String      message;
    // constructor + getters...
}
```

Alert System — Chain of Responsibility (continued) JAVA

```java
// AlertManager.java — Spring collects ALL AlertStrategy beans automatically
@Component
public class AlertManager {
    private final List<AlertStrategy> strategies;
    public AlertManager(List<AlertStrategy> strategies) {
        this.strategies = strategies;
    }
    public List<AlertResult> evaluate(List<Equipment> equipment) {
        return equipment.stream()
            .flatMap(e -> strategies.stream()
                .filter(s -> s.supports(e))
                .map(s -> s.evaluate(e)))
            .collect(Collectors.toList());
    }
}

// In AlertMenu — "View all alerts": fetch all equipment, evaluate, display
List<Equipment> allEquipment = equipmentService.findByUserId(userId);
List<AlertResult> results = alertManager.evaluate(allEquipment);
results.forEach(r -> System.out.printf(
    "[%s] %s %s — %s%n", r.getAlertType().getDisplayName(),
    r.getEquipment().getBrand(), r.getEquipment().getName(), r.getMessage()));

// In AlertMenu — "Overdue return alerts": filter to type only
results.stream()
    .filter(r -> r.getAlertType() == AlertType.OVERDUE_RETURN)
    .forEach(r -> System.out.println(r.getMessage()));
```

08 / NAVIGATION

Menu Navigation Pattern

Every menu class implements the `IMenu` interface, which declares `show()` and `printOptions()`. The body of `show()` is a `do-while` loop: print options → read integer → switch dispatch → exit on `0`. Returning from `show()` pops back to the caller automatically — no explicit stack needed. Spring injects all services; menu classes own only I/O logic.

Menu Navigation Pattern JAVA

```java
// IMenu.java — implemented by all menu classes
public interface IMenu {
    void show();
    void printOptions();
}

// RentalMenu.java — typical menu implementation
@Component
public class RentalMenu implements IMenu {

    private final RentalAgreementService rentalService;

    public RentalMenu(RentalAgreementService rentalService) {
        this.rentalService = rentalService;
    }

    @Override
    public void printOptions() {
        System.out.println("1. Check out equipment");
        System.out.println("2. Check in / return");
        System.out.println("3. View active rentals");
        System.out.println("4. View overdue rentals");
        System.out.println("5. View rental history");
        System.out.println("0. Back");
    }

    @Override
    public void show() {
        int choice;
        do {
            printOptions();
            choice = InputHandler.readInt();  // static — no injection needed
            switch (choice) {
                case 1 -> checkOutEquipment();
                case 2 -> checkInEquipment();
                case 3 -> viewActiveRentals();
                case 4 -> viewOverdueRentals();
                case 5 -> viewRentalHistory();
            }
        } while (choice != 0);
    }   // returning here pops back to MainMenu automatically
}
```

Sample Data

Type	Item	Status	Key Fields
POWER_TOOL	DeWalt 20V Hammer Drill	AVAILABLE	18V · Battery-powered · DRILL · GOOD condition
POWER_TOOL	Milwaukee Circular Saw	RENTED	120V corded · SAW · Due back 2026-08-04
HEAVY_MACHINERY	Bobcat E35 Mini Excavator	RENTED	412 engine hrs · DIESEL · Cert required · Overdue (triggers alert)
HEAVY_MACHINERY	Honda EU7000is Generator	AVAILABLE	210 engine hrs · GASOLINE · No cert required
EVENT_EQUIPMENT	20×40 Frame Tent	AVAILABLE	TENT · Setup required · Qty: 1 · Warranty exp. 2026-08-12 (triggers alert)
EVENT_EQUIPMENT	Round Table + Chair Bundle	AVAILABLE	TABLE_CHAIR · Qty: 8 bundles of 10 · No setup required